

ATIVIDADE:: DETECÇÃO DE CODE SMELLS

1. Recupere seus projetos anteriores envolvendo Programação Orientada a Objetos (mas não restrito à OO), identifique e liste quais Code Smells estavam presentes.

```
8      public Account(int number, String name, double initialDeposit) {
12    }
13
14    public Account(int number, String name) {
15        this.number = number;
16        this.name = name;
17    }
18
19    public int getNumber() {
20        return number;
21    }
22
23    public String getName() {
24        return name;
25    }
26
27    public void setName(String name) {
28        this.name = name;
29    }
30
31    public double getBalance() {
32        return balance;
33    }
34
35    public void deposit(double value) {
36        this.balance += value;
37    }
38
39    public void withdraw(double value) {
40        this.balance -= value + 5.00;
41    }
42
43    public String toString() {
44        return "Dados da conta - Número: "
45            + number + " | Titular: "
46            + name
47            + " | Saldo: "
48            + String.format("%.2f", balance);
49    }
50 }
```

Números Mágicos: O uso do valor fixo 5.00 diretamente no método de saque, sem uma constante explicativa.

Duplicação de Código: Repetição das mesmas atribuições de variáveis em diferentes construtores.

Obsessão por Primitivos: Utilização do tipo double para lidar com valores monetários, o que pode causar erros de precisão.

Falta de Validação: Os métodos de depósito e saque permitem estados inconsistentes, como valores negativos ou saldos descobertos sem verificação.

2. Considerando o que aprendeu até aqui, como poderia melhorar esses projetos e remover os Code Smells.

Composição: Uso de Extract Method para reduzir métodos longos e Extract Variable para simplificar expressões complexas.

Organização: Uso de Move Method ou Extract Class para redistribuir responsabilidades e reduzir acoplamento.

Generalização: Uso de Pull Up/Push Down Method para organizar comportamentos na hierarquia de classes.

3. Selecione dois a três projetos de software populares e registrados no Git. Pesquise em seu histórico referências para Code Smells, indicando o link e trecho de código em que ocorreram.

Link: <https://github.com/apache/commons-lang/pull/1512>

```
259 +
260 +   while (lhsClazz.getSuperclass() != null && lhsClazz != reflectUpToClass) {
261 +       lhsClazz = lhsClazz.getSuperclass();
262     reflectionAppend(lhs, rhs, lhsClazz, compareToBuilder, compareTransients, excludeFields);
-   while (lhsClazz.getSuperclass() != null && lhsClazz != reflectUpToClass) {
-       lhsClazz = lhsClazz.getSuperclass();
-       reflectionAppend(lhs, rhs, lhsClazz, compareToBuilder, compareTransients, excludeFields);
-   }
-   return compareToBuilder.toComparison();
263 }
264
265 +   return compareToBuilder.toComparison();
266 + }
267 +
```

Link: https://github.com/apache/commons-collections/pull/471?utm_source=chatgpt.com

```

@@ -106,7 +107,7 @@ class Indices {
106 107     private int size;
107 108
108 109     boolean add(final int index) {
109 110         - data = IndexUtils.ensureCapacityForAdd(data, size);
110 111         + data = ensureCapacityForAdd(data, size);
110 112         data[size++] = index;
111 113         return true;
112 114     }
@@ -173,4 +174,20 @@ public IndexProducer uniqueIndices() {
173 174     }
174 175     };
175 176 }

```

4. Para o exemplo anterior, observe como o problema foi resolvido.

O problema foi a duplicação de loops e fluxo de controle confuso, com return dentro do laço. A correção removeu o loop duplicado e simplificou a estrutura, tornando o código mais claro e correto.

O principal erro foi a chamada duplicada de `ensureCapacityForAdd`. A solução foi eliminar a redundância e centralizar a lógica em um único método, reduzindo acoplamento e melhorando a organização do código.

5. Pesquise e liste ao menos três ferramentas que ajudem a detectar Code Smells no código. Faça uma tabela comparativa indicando quais Code Smells podem ser detectados.

Ferramenta	Code Smells detectados
SonarQube	Código duplicado; Método longo; Classe muito grande; Muitos parâmetros
PMD	Código duplicado; Método longo; Condicionais complexas
JDeodorant	God Class; Long Method; Feature Envy; Código duplicado